

Code-Layout, Readability and Reusability

Date	03 July 2026
Team ID	PNT2026TMID-EG01
Project Name	EduGenie - AI-Powered Educational Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-style 4-space indentation followed across all FastAPI and JS files
2	Proper File Structure	Files and folders are logically organized	Yes	Backend split into models/, routers/, schemas/, services/; frontend split into css/, js/, img/
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names like user_id, quiz_score, roadmap_topic used consistently
4	Function / Method Names	Functions are descriptively named	Yes	e.g., generate_quiz(), summarize_text(), get_learning_roadmap() in gemini_service.py
5	Code Comments	Inline and block comments explain logic	Partial	Core AI service logic is commented; some router/schema files have minimal comments
6	Modular Design	Code is split into reusable functions/modules	Yes	services/gemini_service.py centralizes all AI calls, reused across ai.py, history.py routers
7	No Redundant Code	Duplicate or unused code is removed	Yes	Common request/response handling consolidated; no duplicate route logic found
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try/except blocks around Gemini API calls and DB operations with proper HTTPException responses

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
------	-------------------------	-----------------------	--------------	---

1	gemini_service.py	Python (Google Generative AI SDK)	Reused across ask-ai, explain, quiz generation, summarization, and roadmap features (ai.py router)	High
2	auth.py (auth utilities)	Python (FastAPI, JWT)	Reused for login, registration, and protected route verification across all routers	High
3	database.py	Python (SQLAlchemy)	Shared DB session/connection logic reused by every model and router	High
4	api.js	JavaScript (Fetch API)	Central API request handler reused across dashboard.js, auth.js, and all frontend pages	High
5	glass.css	CSS	Shared glassmorphism UI styling reused across index, dashboard, quiz, and roadmap pages	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of backend (models/routers/schemas/services) and frontend (css/js/html) layers
Readability	4	Descriptive naming throughout; minor improvement possible in router-level comments
Reusability	5	Centralized gemini_service.py and api.js maximize reuse across all AI-powered features
Documentation / Comments	4	README.md and inline comments present; could add more docstrings for service functions
Overall Score	4.5	Well-structured, maintainable, and extensible codebase